



Exclusões, relacionamentos e integridade dos dados

Como decidir o que acontece com as vendas quando um veículo, cliente, marca ou cidade é excluído no StockCar Manager.

Objetivo do material

Explicar os conceitos antes da implementação, mostrar exemplos pequenos em C# e apresentar uma estratégia segura para preservar o histórico comercial da aplicação.

Projeto: StockCar Manager

Tecnologias: ASP.NET Core MVC, Entity Framework Core e PostgreSQL

Material de apoio para iniciantes

Sumário

1. O problema que estamos resolvendo	4
Relações existentes no projeto	4
2. Vocabulário essencial	4
Entidade	4
Chave primária	4
Chave estrangeira	4
Entidade principal e dependente	4
Integridade referencial	5
Migration	5
3. Os comportamentos de exclusão	6
4. Opção A: exclusão em cascata	6
Lendo o código linha por linha	6
Controller com Cascade	7
5. Opção B: impedir a exclusão	8
Validação amigável no controller	8
Exclusão depois da validação	8
6. Tratando erros do banco	8
7. Opção C: exclusão manual dos dependentes	9
Por que usar um único SaveChangesAsync?	9
8. Opção D: SetNull	9
9. Opção E: exclusão lógica	9
Controller	10
Filtro global	10
10. Estratégia recomendada para o StockCar Manager	11
11. Configurando todos os relacionamentos	11
12. Atualizando o banco com migrations	12
Passo 1: modificar o relacionamento	12
Passo 2: criar a migration	12

Passo 3: revisar a migration	12
Passo 4: aplicar no PostgreSQL	12
Passo 5: testar	12
13. Mostrando mensagens na interface	12
Exemplo na Razor View	12
14. Como pensar antes de escolher Cascade	13
Exemplo de efeito em cadeia	13
15. Checklist de implementação	13
16. Exemplos de cenários de teste	14
17. Resumo final	15
Próximo passo sugerido	15

1. O problema que estamos resolvendo

Em um banco relacional, uma venda não guarda todos os dados do veículo dentro dela. Em vez disso, a venda armazena o identificador do veículo, chamado de chave estrangeira. Essa referência permite saber exatamente qual veículo participou da negociação.

```
public class Venda
{
    public int Id { get; private set; }

    public int VeiculoId { get; private set; }
    public Veiculo Veiculo { get; private set; }
}
```

Ideia principal

VeiculoId guarda o número do veículo. Veiculo permite acessar o objeto relacionado no código C#. O banco garante que a referência continue válida.

Relações existentes no projeto

- uma cidade pode estar relacionada a vários clientes;
- uma marca pode estar relacionada a vários veículos;
- um cliente pode estar relacionado a várias vendas;
- um veículo está referenciado por uma ou mais vendas no esquema atual.

Registro principal	Registro dependente	Chave estrangeira
Cidade	Cliente	CidadeId
Marca	Veículo	MarcaId
Cliente	Venda	ClientId
Veículo	Venda	VeiculoId

2. Vocabulário essencial

Entidade

Uma classe C# que representa algo armazenado no banco, como Cidade, Cliente, Veiculo ou Venda.

Chave primária

Campo que identifica um registro de forma única. No projeto, esse campo normalmente é o Id.

Chave estrangeira

Campo que aponta para a chave primária de outra tabela. VeiculoId, por exemplo, aponta para o Id de Veiculo.

Entidade principal e dependente

Na relação entre Veiculo e Venda, o veículo é o principal e a venda é a dependente, pois Venda precisa da chave do veículo para manter a associação.

Integridade referencial

É a garantia de que o banco não manterá uma referência quebrada. Uma venda não pode apontar para um veículo que não existe, salvo quando o relacionamento for opcional e a chave puder ser nula.

Migration

É uma descrição versionada das mudanças do banco. Quando o relacionamento muda no código, uma migration altera a regra correspondente no PostgreSQL.

3. Os comportamentos de exclusão

Comportamento	O que acontece	Uso típico
Cascade	Exclui também os registros dependentes.	Dados sem valor fora do registro principal.
Restrict	Bloqueia a exclusão se houver dependentes.	Histórico e dados comerciais.
NoAction	Deixa o banco validar e bloquear a operação.	Semelhante a Restrict no PostgreSQL.
SetNull	Mantém o dependente e limpa a chave estrangeira.	Relacionamentos realmente opcionais.
Exclusão lógica	Mantém a linha e marca o registro como inativo.	Auditoria e recuperação futura.

Pergunta que orienta a decisão

Se o registro principal desaparecer, o dependente perde completamente o sentido? Se a resposta for não, apagar em cascata provavelmente não é a melhor escolha.

4. Opção A: exclusão em cascata

Com Cascade, apagar o veículo também apaga as vendas que possuem aquele VeiculoId. O PostgreSQL realiza essa operação por causa da regra gravada na chave estrangeira.

```
modelBuilder.Entity<Venda>()  
    .HasOne(venda => venda.Veiculo)  
    .WithMany()  
    .HasForeignKey(venda => venda.VeiculoId)  
    .OnDelete(DeleteBehavior.Cascade);
```

Lendo o código linha por linha

- Entity<Venda> informa qual entidade está sendo configurada;
- HasOne informa que cada venda possui um veículo;
- WithMany informa que o lado principal pode ter vários dependentes;
- HasForeignKey seleciona VeiculoId como chave estrangeira;
- OnDelete escolhe o comportamento quando o veículo for excluído.

Controller com Cascade

```
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var veiculo = await _context.Veiculos.FindAsync(id);

    if (veiculo != null)
    {
        _context.Veiculos.Remove(veiculo);
        await _context.SaveChangesAsync();
    }

    return RedirectToAction(nameof(Index));
}
```

O controller não precisa carregar as vendas para que a cascata ocorra. A exclusão dos dependentes é responsabilidade da regra do banco.

Risco para este projeto

Excluir uma venda remove data, cliente, vendedor e valores usados no histórico e no dashboard. Por isso, Cascade é simples, mas perigoso para dados comerciais.

5. Opção B: impedir a exclusão

Com `Restrict`, um veículo sem vendas pode ser excluído. Quando existe uma venda relacionada, o banco recusa a operação. Essa é uma estratégia apropriada para preservar o histórico.

```
modelBuilder.Entity<Venda>()  
    .HasOne(venda => venda.Veiculo)  
    .WithMany()  
    .HasForeignKey(venda => venda.VeiculoId)  
    .OnDelete(DeleteBehavior.Restrict);
```

Validação amigável no controller

Antes de mandar o `DELETE` ao banco, o controller pode verificar se há vendas. Isso permite mostrar uma mensagem compreensível ao usuário.

```
var possuiVendas = await _context.Vendas  
    .AnyAsync(venda => venda.VeiculoId == id);  
  
if (possuiVendas)  
{  
    TempData["Erro"] =  
        "O veículo possui uma venda e não pode ser excluído.";   
  
    return RedirectToAction(nameof(Index));  
}
```

`AnyAsync` não traz todas as vendas para a memória. A consulta apenas pergunta ao banco se existe pelo menos um registro correspondente.

Exclusão depois da validação

```
var veiculo = await _context.Veiculos.FindAsync(id);  
  
if (veiculo != null)  
{  
    _context.Veiculos.Remove(veiculo);  
    await _context.SaveChangesAsync();  
}
```

Duas camadas de proteção

O controller oferece uma boa mensagem. A restrição no banco oferece a garantia final, inclusive quando duas pessoas usam o sistema ao mesmo tempo.

6. Tratando erros do banco

Mesmo depois da consulta com `AnyAsync`, outro usuário pode registrar uma venda antes da exclusão terminar. Esse cenário é chamado de condição de corrida. O banco ainda precisa proteger os dados.

```
try  
{  
    _context.Veiculos.Remove(veiculo);  
    await _context.SaveChangesAsync();  
}  
catch (DbUpdateException)  
{  
    TempData["Erro"] =  
        "Não foi possível excluir o veículo porque ele está em uso.";   
  
    return RedirectToAction(nameof(Index));  
}
```


DbUpdateException representa uma falha ocorrida enquanto o Entity Framework tentava salvar mudanças. Em uma aplicação maior, o erro também deve ser registrado em log.

7. Opção C: exclusão manual dos dependentes

Outra possibilidade é carregar as vendas, marcar todas para exclusão e depois remover o veículo. Isso oferece controle explícito, mas continua apagando o histórico.

```
var veiculo = await _context.Veiculos.FindAsync(id);

var vendas = await _context.Vendas
    .Where(venda => venda.VeiculoId == id)
    .ToListAsync();

_context.Vendas.RemoveRange(vendas);
_context.Veiculos.Remove(veiculo);

await _context.SaveChangesAsync();
```

Por que usar um único SaveChangesAsync?

O Entity Framework normalmente agrupa essas mudanças em uma transação. Se uma etapa falhar, o banco pode desfazer toda a operação e evitar que apenas uma parte dos dados seja apagada.

Quando faz sentido

Use a exclusão manual quando os dependentes realmente precisam ser apagados e você deseja executar ações adicionais, como gerar logs ou remover arquivos relacionados.

8. Opção D: SetNull

SetNull mantém a venda, mas define VeiculoId como nulo. Para isso, a propriedade precisa aceitar null. O ponto de interrogação transforma int e Veiculo em tipos anuláveis.

```
public int? VeiculoId { get; private set; }
public Veiculo? Veiculo { get; private set; }

modelBuilder.Entity<Venda>()
    .HasOne(venda => venda.Veiculo)
    .WithMany()
    .HasForeignKey(venda => venda.VeiculoId)
    .OnDelete(DeleteBehavior.SetNull);
```

Essa opção não combina muito bem com Venda, porque uma venda sem veículo perde uma informação central. Ela é mais adequada para relações que sejam verdadeiramente opcionais.

9. Opção E: exclusão lógica

Na exclusão lógica, a linha continua no banco. O sistema apenas marca o veículo como inativo. Essa estratégia preserva o histórico e permite recuperação futura.

```
public class Veiculo
{
    public bool Ativo { get; private set; } = true;

    public void Desativar()
    {
        Ativo = false;
    }
}
```

Controller

```
var veiculo = await _context.Veiculos.FindAsync(id);

if (veiculo != null)
{
    veiculo.Desativar();
    await _context.SaveChangesAsync();
}
```

Filtro global

```
modelBuilder.Entity<Veiculo>()
    .HasQueryFilter(veiculo => veiculo.Ativo);
```

Depois do filtro global, consultas normais retornam somente veículos ativos. Para uma tela administrativa que também mostre os inativos, pode-se usar `IgnoreQueryFilters`.

```
var todos = await _context.Veiculos
    .IgnoreQueryFilters()
    .ToListAsync();
```

10. Estratégia recomendada para o StockCar Manager

Situação	Comportamento recomendado
Veículo nunca vendido	Permitir exclusão física.
Veículo com venda	Bloquear exclusão ou desativar.
Veículo fora do estoque	Desativar e preservar o histórico.
Venda digitada incorretamente	Corrigir ou cancelar de forma controlada.
Venda concluída	Preservar como registro histórico.
Marca com veículos	Bloquear a exclusão da marca.
Cidade com clientes	Bloquear a exclusão da cidade.
Cliente com vendas	Bloquear ou desativar o cliente.

Recomendação prática

Use Restrict nos relacionamentos que envolvem vendas. Considere exclusão lógica para veículos e clientes. Preserve vendas concluídas, pois elas alimentam o histórico e os indicadores do sistema.

11. Configurando todos os relacionamentos

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Cliente>()
        .HasOne(cliente => cliente.Cidade)
        .WithMany()
        .HasForeignKey(cliente => cliente.CidadeId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Veiculo>()
        .HasOne(veiculo => veiculo.Marca)
        .WithMany()
        .HasForeignKey(veiculo => veiculo.MarcaId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Venda>()
        .HasOne(venda => venda.Cliente)
        .WithMany()
        .HasForeignKey(venda => venda.ClienteId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Venda>()
        .HasOne(venda => venda.Veiculo)
        .WithMany()
        .HasForeignKey(venda => venda.VeiculoId)
        .OnDelete(DeleteBehavior.Restrict);
}
```

A configuração deve ficar no método `OnModelCreating` da classe `ApplicationDbContext`. Ela substitui o comportamento que o Entity Framework escolheria apenas por convenção.

12. Atualizando o banco com migrations

Modificar o código do ApplicationDbContext não altera sozinho um banco já criado. É preciso gerar e aplicar uma migration.

Passo 1: modificar o relacionamento

Adicione o OnDelete desejado dentro de OnModelCreating e salve o arquivo.

Passo 2: criar a migration

```
dotnet ef migrations add RestringirExclusoes --project GerenciadorVendaVeiculos/GerenciadorVendaVeiculos.csproj
```

Passo 3: revisar a migration

Abra o arquivo gerado e confira se a chave estrangeira foi recriada com o comportamento esperado. Esse passo ajuda a detectar alterações não planejadas.

Passo 4: aplicar no PostgreSQL

```
dotnet ef database update --project GerenciadorVendaVeiculos/GerenciadorVendaVeiculos.csproj
```

Passo 5: testar

- crie um veículo sem venda e confirme que ele pode ser excluído;
- crie outro veículo e registre uma venda;
- tente excluir o veículo vendido e confira se a operação é bloqueada;
- confirme que a venda continua no banco;
- confira se a mensagem apresentada é compreensível.

Antes de migrations destrutivas

Faça backup quando houver dados importantes. Leia a migration gerada e nunca teste uma mudança de exclusão pela primeira vez diretamente em produção.

13. Mostrando mensagens na interface

TempData transporta uma mensagem durante o redirecionamento. O controller define a mensagem e a view seguinte a exibe.

```
TempData["Erro"] =  
    "O veículo possui uma venda e não pode ser excluído.";   
  
return RedirectToAction(nameof(Index));
```

Exemplo na Razor View

```
@if (TempData["Erro"] is string mensagem)  
{  
    <div class="rounded-md bg-red-50 p-4 text-red-700">  
        @mensagem  
    </div>  
}
```

A mensagem existe somente até ser lida. Isso combina com o padrão Post/Redirect/Get usado depois de formulários.

14. Como pensar antes de escolher Cascade

- O registro dependente possui valor histórico, fiscal ou de auditoria?
- O dashboard ou algum relatório depende desse registro?
- A exclusão poderia atravessar vários relacionamentos e apagar mais do que o esperado?
- O usuário entende claramente o impacto antes de confirmar?
- Existe backup ou possibilidade de recuperação?
- Uma desativação atenderia melhor à necessidade?

Exemplo de efeito em cadeia

```
Excluir Marca
  -> exclui Veículos da marca
      -> exclui Vendas dos veículos
          -> altera totais e apaga histórico
```

Mesmo que cada relacionamento pareça simples isoladamente, uma sequência de cascatas pode produzir uma exclusão ampla. Por isso, entidades cadastrais e transacionais merecem políticas diferentes.

15. Checklist de implementação

- definir por escrito o comportamento esperado para cada relacionamento;
- configurar DeleteBehavior no ApplicationDbContext;
- criar e revisar a migration;
- aplicar a migration em um banco de desenvolvimento;
- consultar dependências no controller para gerar mensagens amigáveis;
- tratar DbUpdateException como proteção adicional;
- mostrar o erro na Razor View;
- testar exclusão com e sem dependentes;
- testar operações simultâneas quando a regra for crítica;
- adicionar testes automatizados para evitar regressões.

16. Exemplos de cenários de teste

Cenário	Ação	Resultado esperado
Veículo sem venda	Excluir veículo	Veículo removido.
Veículo vendido	Excluir veículo	Operação bloqueada e venda preservada.
Marca com veículos	Excluir marca	Operação bloqueada.
Cidade com clientes	Excluir cidade	Operação bloqueada.
Cliente com vendas	Excluir cliente	Operação bloqueada.
Registro inexistente	Excluir por Id inválido	Retorno seguro, sem falha geral.
Conflito simultâneo	Vender e excluir ao mesmo tempo	Banco preserva a integridade.

17. Resumo final

Escolha	Resumo
Cascade	Apaga dependentes automaticamente. Simples, mas pode remover histórico.
Restrict	Impede exclusões que quebrariam relações. Recomendado para vendas.
NoAction	Deixa a restrição ser aplicada pelo banco.
SetNull	Mantém o dependente sem o principal. Exige chave estrangeira anulável.
Exclusão manual	O código controla a ordem e os registros removidos.
Exclusão lógica	Preserva a linha e marca o registro como inativo.

Conclusão

Para o StockCar Manager, o caminho mais seguro é preservar as vendas, restringir a exclusão de registros usados por elas e utilizar desativação quando algo não deve mais aparecer na operação diária.

Próximo passo sugerido

Implementar primeiro a restrição Veiculo -> Venda em uma branch de estudo, criar a migration, testar os dois cenários principais e somente depois repetir o padrão para Cliente, Marca e Cidade.